

YouTube URL + Transcript System

A simple tutorial workflow for copying messy YouTube video blocks, cleaning hidden URLs in Google Sheets, and turning CSV files into clean transcript text files.

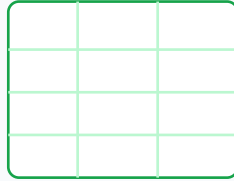
YouTube blocks



Copy video blocks

Messy YouTube blocks are OK.

Google Sheet



Clean in Sheets

Extract hidden URLs.

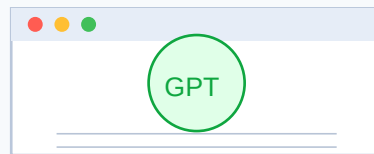
WhisperFlow



Use voice prompts

Generate scripts faster.

ChatGPT Desktop App



ChatGPT floating window

Prompt and adjust the code while recording.

yt-dlp

macOS tool

YT-DLP tool

Pulls subtitles and metadata.

Terminal

```
$ python3  
tools/script.py
```

Terminal command

Run one script.

Transcript .txt



Output file

One clean transcript per channel.

Final tutorial order

- Copy all video blocks from the YouTube channel page.
- Clean hidden URLs in Google Sheets.
- Save the cleaned URL list as a CSV file.
- Install and unblock YT-DLP on macOS.
- Create the Python transcript script with AI.
- Open Terminal and run the transcript script.
- Read the finished transcript text file in the output folder.

Clean Output Preview

This is what the final transcript file should look like. Show this near the beginning of the YouTube tutorial so viewers understand the goal before the setup starts.

```
Alex Hormozi
=====

Video 1
Date: 2024-06-18
Title: I Built a $100M Business From Scratch

URL: https://www.youtube.com/watch?v=abc123
Duration: 18:42
Views: 124,532
Likes: 4,821


Full transcript in one paragraph. The entire video transcript continues here without timestamps...


Video 2
Date: 2024-06-12
Title: The Simple Rule That Changed My Business

URL: https://www.youtube.com/watch?v=xyz456
Duration: 11:07
Views: 88,201


Full transcript in one paragraph...
```

What You Need Before Starting

YouTube blocks



Copy video blocks
Messy YouTube blocks are OK.

Google Sheet



Clean in Sheets
Extract hidden URLs.

WhisperFlow



Use voice prompts
Generate scripts faster.

ChatGPT Desktop App



ChatGPT floating window
Use the desktop app while building the script.

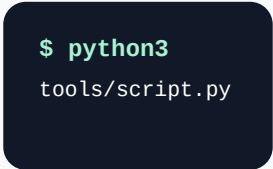
The logo for yt-dlp, featuring the text 'yt-dlp' in a bold, teal, sans-serif font.

macOS tool

YT-DLP tool

Pulls subtitles and metadata.

Terminal

A screenshot of a terminal window with a dark background. It shows the command '\$ python3 tools/script.py' in a light green monospace font.

Terminal command

Run one script.

Transcript .txt



Output file

One clean transcript per channel.

- A YouTube channel page you want to extract video URLs from.
- Google Sheets for cleaning the hidden hyperlinks.
- WhisperFlow or any voice dictation tool if you want to generate code by speaking prompts.
- A Mac with Terminal access.
- The yt-dlp_macos download placed inside the tools folder.
- A Transcript System folder with input, output, and tools inside it.

Step 1 - Copy All Video Blocks

Goal: copy messy YouTube video blocks, not perfect URLs.

On the YouTube channel Videos tab, scroll down to the older videos first. Then copy upward in chunks. It is fine if the video thumbnail, title, views, upload date, and extra text all get copied together. The important part is that the video title still carries the hidden YouTube hyperlink.

- Open the YouTube channel and go to the Videos tab.
- Scroll down until you reach the older videos you want to include.
- Select a group of video blocks from bottom upward.
- Copy the messy selection, even if it includes thumbnails, titles, views, and dates.
- Paste the messy content into Google Sheets column A.

Tip: do not worry if it looks ugly in the sheet. The Apps Script will extract the hidden links from the titles and delete everything else.

Step 2 - Clean Hidden URLs in Google Sheets

The Google Apps Script reads column A, extracts the real rich text hyperlink from each YouTube title, removes duplicates, and rewrites clean URLs back into column A.

Short WhisperFlow Prompt

Write me a simple Google Apps Script for Google Sheets. Column A has messy YouTube pasted content where the real URL is hidden as a rich text hyperlink behind the title. Add a Tools menu item called Clean YouTube URLs that runs `cleanYouTubeHyperlinksInColumnA`, and auto run when column A below row 1 is edited. Read column A from row 2 using `getRichTextValues`, first try `rich.getLinkUrl`, then `rich.getRuns` and `runs[j].getLinkUrl`. Keep only `youtube.com`, `www.youtube.com`, and `youtu.be` links, remove duplicates, clear column A, rewrite the clean URLs back into column A, set A1 to URL, format black size 11 no underline, white background, clip overflow with `SpreadsheetApp.WrapStrategy.CLIP`, and delete extra empty rows. Keep the code simple.

How to Add the Apps Script

- Open the Google Sheet with the messy pasted video blocks.
- Click Extensions, then Apps Script.
- Delete the starter code.
- Paste the code below.
- Save the project.
- Refresh the Google Sheet.
- Use Tools, then Clean YouTube URLs.

Google Apps Script Code

```
// Adds a custom menu when the sheet opens
function onOpen() {
  SpreadsheetApp.getUi()
    .createMenu("Tools")
    .addItem("Clean YouTube URLs", "cleanYouTubeHyperlinksInColumnA")
    .addToUi();
}

// Auto-runs when any cell is edited in column A below the header
function onEdit(e) {
  const sheet = e.range.getSheet();
  const col = e.range.getColumn();
  const row = e.range.getRow();

  if (col === 1 && row > 1) {
    cleanYouTubeHyperlinksInColumnA();
  }
}

function cleanYouTubeHyperlinksInColumnA() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();
  const startRow = 2;
  const column = 1;
  const lastRow = sheet.getLastRow();

  const headerCell = sheet.getRange(1, column);
  headerCell.setValue("URL");
  headerCell
    .setFontWeight("bold")
    .setFontSize(11)
    .setFontColor("black")
    .setFontLine("none")
    .setBackground("white");

  if (lastRow < startRow) return;

  const numRows = lastRow - startRow + 1;
  const range = sheet.getRange(startRow, column, numRows, 1);
  const richValues = range.getRichTextValues();

  const seen = new Set();
  const cleaned = [];

  for (let i = 0; i < richValues.length; i++) {
    const rich = richValues[i][0];
    if (!rich) continue;

    let url = rich.getLinkUrl();

    if (!url) {
      const runs = rich.getRuns();
      for (let j = 0; j < runs.length; j++) {
        url = runs[j].getLinkUrl();
        if (url) break;
      }
    }

    if (!url) continue;

    url = url.trim();

    const isYouTube =
      /^https?:\/\/(www\.)?(youtube\.com|youtu\.be)\//i.test(url);

    if (!isYouTube) continue;

    if (!seen.has(url)) {
      seen.add(url);
      cleaned.push([url]);
    }
  }

  range.clearContent();

  if (cleaned.length > 0) {
    const outputRange = sheet.getRange(startRow, column, cleaned.length, 1);
    outputRange.setValues(cleaned);
    outputRange
      .setFontColor("black")
      .setFontLine("none")
      .setFontWeight("normal")
      .setFontSize(11)
      .setBackground("white");
  }
}
```

```
        .setWrapStrategy(SpreadsheetApp.WrapStrategy.CLIP);
    }

    sheet.getRange(1, column, sheet.getMaxRows(), 1)
        .setBackground("white")
        .setWrapStrategy(SpreadsheetApp.WrapStrategy.CLIP);

    const currentMaxRows = sheet.getMaxRows();
    const neededRows = startRow - 1 + cleaned.length;
    const extraRows = currentMaxRows - neededRows;

    if (extraRows > 0) {
        sheet.deleteRows(neededRows + 1, extraRows);
    }
}
```

Step 3 - Save the Clean URLs as CSV

- Make sure cell A1 says URL.
- Make sure every video URL is underneath it in column A.
- Name the Google Sheet tab or file after the creator.
- Download the sheet as a CSV file.
- Place the CSV file inside Transcript System/input.

File > Download > Comma Separated Values (.csv)

Example file name:

Alex Hormozi.csv

Step 4 - Install YT-DLP

The transcript script expects yt-dlp to be inside the tools folder as a folder called yt-dlp_macos, with the executable inside it.

Correct folder structure

```
Transcript System/  
  input/  
    Alex Hormozi.csv  
    Other Creator.csv  
  
  output/  
    Alex Hormozi_transcript.txt  
    Other Creator_transcript.txt  
  
  tools/  
    transcribe_youtube_csvs.py  
  
  yt-dlp_macos/  
    yt-dlp_macos  
    _internal/
```

The important path is:

tools/yt-dlp_macos/yt-dlp_macos

That is the actual executable file the Python script runs.

Step 5 - Create the Python Code with AI

Use this short prompt if you want ChatGPT to regenerate the Python script from scratch.

Create a Python script for a macOS YouTube transcript system. Folder structure: Transcript System with input, output, and tools. Inside tools there is transcribe_youtube_csvs.py and a yt-dlp_macos folder containing the yt-dlp_macos executable. The script should read every CSV in input where column A header is URL, skip a CSV if output already has Creator_transcript.txt, use yt-dlp to pull metadata and English subtitles, then save one transcript text file per CSV in output. At the top include the channel name. For each video include Video number, Date, Title, URL, Duration, Views, Likes if available, then three blank lines before the full transcript in one paragraph. Keep the code simple.

Create the Python File in Terminal

Open Terminal in the main Transcript System folder, then run:

```
touch tools/transcribe_youtube_csvs.py
open -e tools/transcribe_youtube_csvs.py
```

- Paste the Python code into TextEdit.
- Press Command + S to save.
- Close TextEdit.
- Return to Terminal.

Python Transcript Script Code

Save this as tools/transcribe_youtube_csvs.py.

```
import csv
import json
import re
import subprocess
import tempfile
from pathlib import Path

TOOLS_DIR = Path(__file__).resolve().parent
BASE_DIR = TOOLS_DIR.parent
INPUT_DIR = BASE_DIR / "input"
OUTPUT_DIR = BASE_DIR / "output"

YT_DLP = TOOLS_DIR / "yt-dlp_macos" / "yt-dlp_macos"

def run_command(command):
    result = subprocess.run(
        command,
        text=True,
        capture_output=True
    )

    if result.returncode != 0:
        raise Exception(result.stderr.strip())

    return result.stdout.strip()

def read_urls_from_csv(csv_file):
    urls = []

    with open(csv_file, "r", encoding="utf-8-sig", newline="") as file:
        reader = csv.DictReader(file)

        for row in reader:
            url = row.get("URL", "").strip()
            if url:
                urls.append(url)

    return urls

def format_date(upload_date):
    if upload_date and len(upload_date) == 8:
        return f"{upload_date[:4]}-{upload_date[4:6]}-{upload_date[6:]}"

    return "Unknown date"

def format_duration(seconds):
    if not seconds:
        return "Unknown duration"

    seconds = int(seconds)
    hours = seconds // 3600
    minutes = (seconds % 3600) // 60
    secs = seconds % 60

    if hours > 0:
        return f"{hours}:{minutes:02d}:{secs:02d}"

    return f"{minutes}:{secs:02d}"

def format_number(number):
    if number is None:
        return None

    return f"{number:,}"

def get_video_info(url):
    output = run_command([
        str(YT_DLP),
        "--dump-json",
        "--skip-download",
        url
    ])

    data = json.loads(output)

    return {
        "title": data.get("title", "Untitled Video"),
        "upload_date": format_date(data.get("upload_date", "")),
        "channel": data.get("channel") or data.get("uploader") or "Unknown Channel",
        "duration": format_duration(data.get("duration")),
        "views": format_number(data.get("view_count")),
        "likes": format_number(data.get("like_count")),
    }
```

```

        "url": url
    }

def download_subtitle(url, temp_dir):
    run_command([
        str(YT_DLP),
        "--skip-download",
        "--write-auto-sub",
        "--write-sub",
        "--sub-lang", "en",
        "--sub-format", "vtt",
        "-o", str(temp_dir / "%(id)s.%(ext)s"),
        url
    ])

    subtitle_files = list(temp_dir.glob("*.vtt"))

    if not subtitle_files:
        return ""

    return subtitle_files[0].read_text(encoding="utf-8", errors="ignore")

def clean_vtt_text(vtt_text):
    lines = vtt_text.splitlines()
    clean_lines = []

    for line in lines:
        line = line.strip()

        if not line:
            continue

        if line == "WEBVTT":
            continue

        if "-->" in line:
            continue

        if line.isdigit():
            continue

        if line.startswith("Kind:") or line.startswith("Language:"):
            continue

        line = re.sub(r"<[^>+>", "", line)
        line = re.sub(r"\s+", " ", line).strip()

        if line:
            clean_lines.append(line)

    transcript = " ".join(clean_lines)
    transcript = re.sub(r"\s+", " ", transcript).strip()

    return transcript

def transcribe_video(url):
    info = get_video_info(url)

    with tempfile.TemporaryDirectory() as temp:
        temp_dir = Path(temp)
        vtt_text = download_subtitle(url, temp_dir)
        transcript = clean_vtt_text(vtt_text)

    if not transcript:
        transcript = "[No English subtitles found for this video.]"

    return info, transcript

def make_video_section(video_number, info, transcript):
    lines = [
        f"Video {video_number}",
        f"Date: {info['upload_date']}",
        f"Title: {info['title']}",
        "",
        f"URL: {info['url']}",
        f"Duration: {info['duration']}",
    ]

    if info["views"]:
        lines.append(f"Views: {info['views']}")

    if info["likes"]:
        lines.append(f"Likes: {info['likes']}")

    lines.append("")

```

```

        lines.append("")
        lines.append("")
        lines.append(transcript)

    return "\n".join(lines)

def process_csv(csv_file):
    creator_name = csv_file.stem
    output_file = OUTPUT_DIR / f"{creator_name}_transcript.txt"

    if output_file.exists():
        print(f"Skipping already completed: {csv_file.name}")
        return

    urls = read_urls_from_csv(csv_file)

    if not urls:
        print(f"No URLs found in: {csv_file.name}")
        return

    print(f"Processing: {csv_file.name}")

    sections = []
    channel_name = creator_name

    for index, url in enumerate(urls, start=1):
        print(f" {index}/{len(urls)}: {url}")

        try:
            info, transcript = transcribe_video(url)

            if index == 1:
                channel_name = info["channel"]

            section = make_video_section(index, info, transcript)
            sections.append(section)

        except Exception as error:
            sections.append(
                f"Video {index}\n"
                f"URL: {url}\n\n"
                f"FAILED VIDEO\n\n"
                f"{error}"
            )

    final_text = (
        f"{channel_name}\n"
        f"{'=' * len(channel_name)}\n"
        "\n\n\n"
        + "\n\n\n\n".join(sections)
    )

    output_file.write_text(final_text, encoding="utf-8")

    print(f"Saved: {output_file.name}")

def main():
    OUTPUT_DIR.mkdir(exist_ok=True)

    csv_files = sorted(INPUT_DIR.glob("*.csv"))

    if not csv_files:
        print("No CSV files found in the input folder.")
        return

    for csv_file in csv_files:
        process_csv(csv_file)

    print("Done.")

if __name__ == "__main__":
    main()

```

Step 6 - Run Terminal and Unblock YT-DLP

Right click the main Transcript System folder and choose New Terminal at Folder. You should run every command from the main Transcript System folder, not from input, output, or tools.

First, give the yt-dlp executable permission and remove macOS quarantine flags:

```
chmod 755 tools/yt-dlp_macos/yt-dlp_macos  
xattr -cr tools/yt-dlp_macos
```

Test yt-dlp directly:

```
./tools/yt-dlp_macos/yt-dlp_macos --version
```

Only continue when this prints a version number.

If yt-dlp cannot run by itself, the transcript script cannot run either.

Step 7 - Run the Transcript Script

Once your CSV files are in input and yt-dlp works, run:

```
python3 tools/transcribe_youtube_csvs.py
```

- The script checks every CSV file in input.
- If an output text file already exists, it skips that CSV.
- If the CSV is new, it downloads subtitles and metadata for every URL.
- It saves one clean transcript text file inside output.

Step 8 - Read the Finished Transcript File

Open the output folder. Each completed channel will have one text file named after the CSV.

```
output/Alex_Hormozi_transcript.txt
```

Each video section includes the video number, date, title, URL, duration, views, likes when available, and then the transcript in one paragraph.

```
Alex Hormozi
=====
```

```
Video 1
Date: 2024-06-18
Title: I Built a $100M Business From Scratch

URL: https://www.youtube.com/watch?v=abc123
Duration: 18:42
Views: 124,532
Likes: 4,821
```

Full transcript in one paragraph. The entire video transcript continues here without timestamps...

```
Video 2
Date: 2024-06-12
Title: The Simple Rule That Changed My Business

URL: https://www.youtube.com/watch?v=xyz456
Duration: 11:07
Views: 88,201
```

Full transcript in one paragraph...

Common Problems

Permission denied

```
chmod 755 tools/yt-dlp_macos/yt-dlp_macos
xattr -cr tools/yt-dlp_macos
```

No CSV files found

Your CSV file is probably not inside the input folder, or it does not end in .csv.

No URLs found

The CSV header must be exactly URL in cell A1.

No English subtitles found

The video may not have English captions or YouTube may not expose auto captions for that video.